

01/24) 05. CSRF(Cross Site Request Forgery) (1)

CSRF

임의 이용자의 권한으로 임의 주소에 HTTP 요청을 보낼 수 있는 취약점

특정 이용자의 정보 입력을 유도해 인증 정보를 얻어 대신 HTTP 요청을 보내는 것

(= 서명된 문서를 위조하는 것)

예시: 송금 기능 코드

- 해당 서비스에서는 로그인 된 상태에서 추가적인 인증 정보 없이 송금이 가능하므로 공격자가 쿠키를 탈취하여 해당 이용자의 권한으로 송금할 수 있게 됨

```
# 이용자가 /sendmoney에 접속했을때 아래와 같은 송금 기능을 웹 서비스가 실행함.
@app.route('/sendmoney')
def sendmoney(name):
    # 송금을 받는 사람과 금액을 입력받음.
    to_user = request.args.get('to')
    amount = int(request.args.get('amount'))

    # 송금 기능 실행 후, 결과 반환
    success_status = send_money(to_user, amount)

    # 송금이 성공했을 때,
    if success_status:
        # 성공 메시지 출력
        return "Send success."
    # 송금이 실패했을 때,
    else:
```

```
# 실패 메시지 출력
return "Send fail."
```

- 인증 정보를 탈취하려면 이용자가 악성 스크립트를 실행하도록 유도해야 함
 - HTML: img 태그 이용

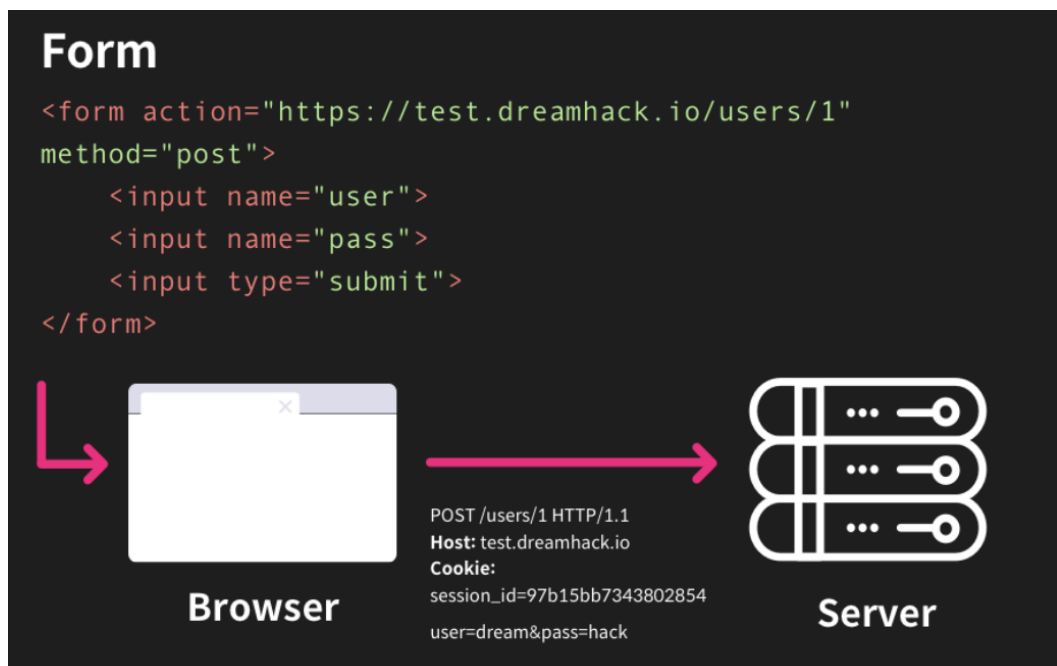
```
# img 태그 이용
<img src='http://bank.dreamhack.io/sendmoney?
to=dreamhack&amount=1337' width=0px height=0px>
```

- HTML : form 태그 이용

글자를 입력하거나 체크하는 등의 데이터 입력 양식

다음과 같은 속성이 있음

- **action** : 폼을 전송한 서버쪽 스트립트 파일 지정
- **name** : 폼 식별하는 이름
- **method** : 폼을 서버에 전송할 http 메소드



- 자바스크립트 이용

```
window.open('http://bank.dreamhack.io/sendmoney?
to=dreamhack&amount=1337');

location.href = 'http://bank.dreamhack.io/sendmoney?
to=dreamhack&amount=1337';
location.replace('http://bank.dreamhack.io/sendmoney?
to=dreamhack&amount=1337');
```

XSS와의 비교

공통점

- 클라이언트 대상, 악성 스크립트 포함된 페이지로 유도해야 함

차이점

- XSS는 쿠키 및 세션 정보를 탈취하는 게 목적인 반면, CSRF는 타 이용자의 권한으로 HTTP 요청을 보내거나 다른 기능을 실행하는 것이 목적

실습

csrf-1

분석

/	인덱스 페이지
/vuln	이용자가 입력한 값 출력하는 페이지
/flag	입력한 url로 이동하게 함
/memo	입력한 것이 메모로 저장됨

/admin/notice_flag	관리자만 접속 가능, flag를 memo에 저장함
--------------------	-----------------------------

- /admin/notice_flag 페이지

userid 인자를 받아 admin인지 확인하고, 맞는 경우에만 플래그를 출력

또한 접근하는 IP가 localhost인지 확인하기 때문에 인증 정보를 탈취해 접근해도 소용없음

⇒ 따라서 관리자로 하여금 직접 notice 페이지에 들어가도록 유도해야 함.

```
@app.route("/admin/notice_flag")
def admin_notice_flag():
    global memo_text
    if request.remote_addr != "127.0.0.1":
        return "Access Denied"
    if request.args.get("userid", "") != "admin":
        return "Access Denied 2"
    memo_text += f"[Notice] flag is {FLAG}\n"
    return "Ok"
```

- Request Bin 기능으로 진짜 CSRF가 일어나는지 확인해보기

아래와 같이 생성된 랜덤 url을 img 태그로 입력해본다.

```

```

그러면 아래와 같이 해당 url로 GET 요청이 보내졌다는 것을 알 수 있다.

즉 관리자가 여기로 유도되었다는 뜻

Request Bin



https://nqrlvbp.request.dreamhack.games

링크생성

시간
01-01 18:33:19

경로
GET /

My Request

Raw Data

IP 23.81.42.210
Method GET
Path /
QueryString

Headers

Accept image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
Accept-Encoding gzip, deflate, br
Host nqrlvbp.request.dreamhack.games
Referer http://127.0.0.1:8000/
Sec-Ch-Ua "Not.A/Brand";v="8", "Chromium";v="114", "Headless Chrome";v="114"
Sec-Ch-Ua-Mobile ?0
Sec-Ch-Ua-Platform "Linux"
Sec-Fetch-Dest image
Sec-Fetch-Mode no-cors
Sec-Fetch-Site cross-site
User-Agent Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/114.0.5735.198 Safari/537.36

Body

8 ※ 최근 100개 항목만 보여집니다.

• 익스플로잇

```
param= 
```

물음표 뒤는 서버에 전달하는 파라미터로, userid가 admin인지 검사하기 때문에 추가해 주어야 함.

이렇게 하면 memo 페이지에 flag가 나타난다.

csrf-2

분석

- /memo와 /admin_notice 페이지가 없고, /login 페이지가 생겼다.
- guest 계정의 비밀번호 주어짐. admin 계정은 비밀번호가 flag로 설정되어 있음.
- flag가 memo에 나타나는 게 아니라, 관리자로 로그인하면 인덱스 페이지에 flag가 나타나는 형식. 이번엔 랜덤한 16진수로 세션이 정해져 있기 때문에 세션을 탈취해 로그인할 수는 없음

취약점

비밀번호를 바꿀 수 있는 페이지가 생김.

`request.args.get` 은 GET 요청에서 인자를 받아오는 것.

아래 코드에서 pw가 GET 요청의 인자를 받아오므로, url에서 물음표 뒤에 pw를 지정해주면 비밀번호가 바뀌게 된다.

```
@app.route("/change_password")
def change_password():
    pw = request.args.get("pw", "")
    session_id = request.cookies.get('sessionid', None)
    try:
        username = session_storage[session_id]
    except KeyError:
        return render_template('index.html', text='please login')

    users[username] = pw
    return 'Done'
```

- 익스플로잇

flag의 빈칸에 /change_password로 이동하게 하는 코드를 넣어준다.

새로 설정할 pw를 입력하면 관리자의 비밀번호가 해당 인자로 바뀐다.
바뀐 비밀번호로 로그인하면 flag가 나타난다.

```

```

vuln(csrf) page

flag

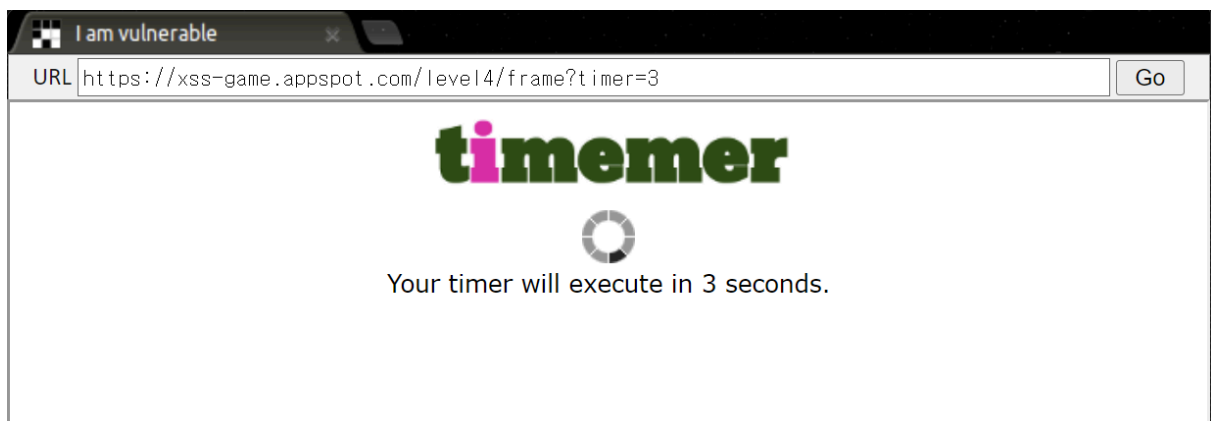
login

Hello admin, flag is
DH{c57d0dc12bb9ff023faf9a0e2b49e470a77271ef}

xss-game level3

- 취약점: url에서 # 다음 표기되는 부분에 사용자가 간섭할 수 있음
- 해당 부분은 코드에서 window.location.hash로 접근
- 익스플로잇 : # 다음 `<img src='' onerror=alert()>` 입력

xss-game level 4



취약점: level.py 에서 startTimer가 변수를 출력하는 부분

```

<br>
<div id="message">Your timer will execute in {{ timer
}} seconds.</div>
```

- 익스플로잇: `' ); alert('` 을 입력하면 startTimer 함수가 닫히고, alert(' }} ') 함수가 실행된다.

```
startTimer('{{ '); alert('}}');
```

xss-game level 5



분석

- description: DOM에 새로운 요소를 삽입하지 않고도 공격을 수행할 수 있다. 즉 이전 처럼 img 태그를 새로 삽입해준다면가 하지 않는다는 것
- level.py 소스를 보면 signup 페이지에서 next로 받은 데이터를 signup.html 코드에 전달한다.


```
# Route the request to the appropriate template
if "signup" in self.request.path:
    self.render_template('signup.html',
        {'next': self.request.get('next')})
elif "confirm" in self.request.path:
    self.render_template('confirm.html',
        {'next': self.request.get('next', 'welcome')})
else:
    self.render_template('welcome.html', {})
```

- signup.html을 보면 next로 받은 인자를 a 태그의 하이퍼링크 속성에 넣어서, Next >>를 누르면 해당 링크로 이동하도록 한다.
- default 값으로 confirm이 지정되어 있어, confirm 페이지를 거쳐 다시 처음의 welcome 페이지로 이동하게 된다.

```
<br><br>
<a href="{{ next }}">Next >></a>
</body>
</html>
```

Hints

1. The title of this level is a hint.
2. It is useful look at the source of the signup frame and see how the URL parameter is used.
3. If you want to make clicking a link execute Javascript (without using the `onclick` handler), how can you do it?
4. If you're really stuck, take a look at this [IETF draft](#)

익스플로잇

- 취약점은 signup 페이지가 href에 인자를 전달하는 부분

- javascript: 속성을 이용한다. 이 속성은 뒤에 오는 스크립트를 실행하게 한다.

```
<a href="javascript:doSomething()">
```

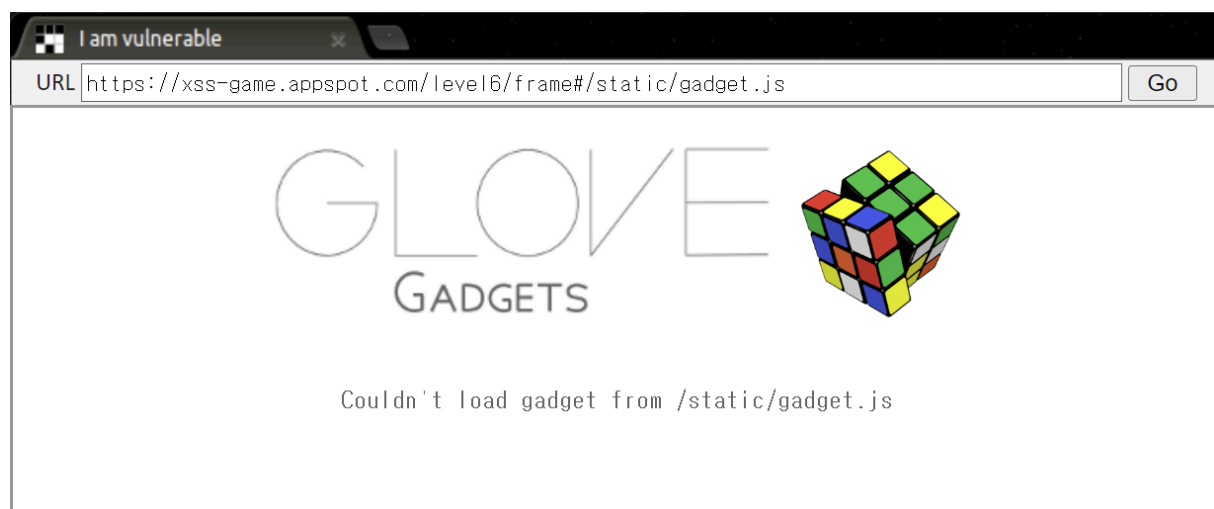
- 따라서 next 파라미터에 다음 값을 넣어준 후 Next를 눌러준다.

```
javascript:alert()
```

의문점: 왜 onclick은 안 되는지? `"onclick=alert()"`

해답: 저렇게 큰따옴표로 href를 닫으면 공백이 전달되어 welcome 페이지로 가게 됨.

xss-game level 6



분석

- url에서 # 뒤에 있는 인자를 전달해 그대로 출력함
- 해당 부분에 alert()를 발생시키는 데이터를 전달하면 됨

힌트

1. See how the value of the location fragment (after `#`) influences the URL of the loaded script.
2. Is the security check on the gadget URL really foolproof?
3. Sometimes when I'm frustrated, I feel like *screaming*...

4. If you can't easily host your own evil JS file, see if google.com/jsapi?callback=foo will help you here.

익스플로잇

찾아보니 두 가지 방법이 있다.

1. 데이터를 URI로 변경하는 Data URI Scheme를 이용
외부 호출 없이 데이터를 임베드 시킬 수 있으며 주로 아이콘 같은 작은 이미지 파일에 사용된다고 함. [참고](#)

- 문법

```
data:[<media type>][;base64],<data>
```

- alert();라는 텍스트를 전달할 것이므로 다음과 같이 입력

```
data:text,alert();
```

2. 힌트 4에 있는 페이지 이용

해당 페이지는 callback으로 전달받은 파라미터 이름의 함수를 실행시킨다.

따라서 callback에 alert 인자를 전달하는 url을 입력하는데, 문제 코드에서 url을 필터링하고 있다.

소문자로 http만 필터링하고 있기 때문에 대문자를 섞어주는 등으로 우회할 수 있다.

```
HTTPS://www.gstatic.com/charts/loader.js?callback=alert
```